UNIVERSITY OF MICHIGAN

Parrot: Efficient Serving of LLM-based Applications with Semantic Variable
&
Towards End-to-End Optimization of LLM-based Applications with Ayo

Nov 6, 2025

Hui Shen, Zesen Zhao, Boyuan Zheng

UNIVERSITY OF MICHIGAN

Parrot: Efficient Serving of LLM-based Applications with Semantic Variable

Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, Lili Qiu

Shanghai Jiao Tong University & Microsoft Research

OSDI 2024 | May 2024

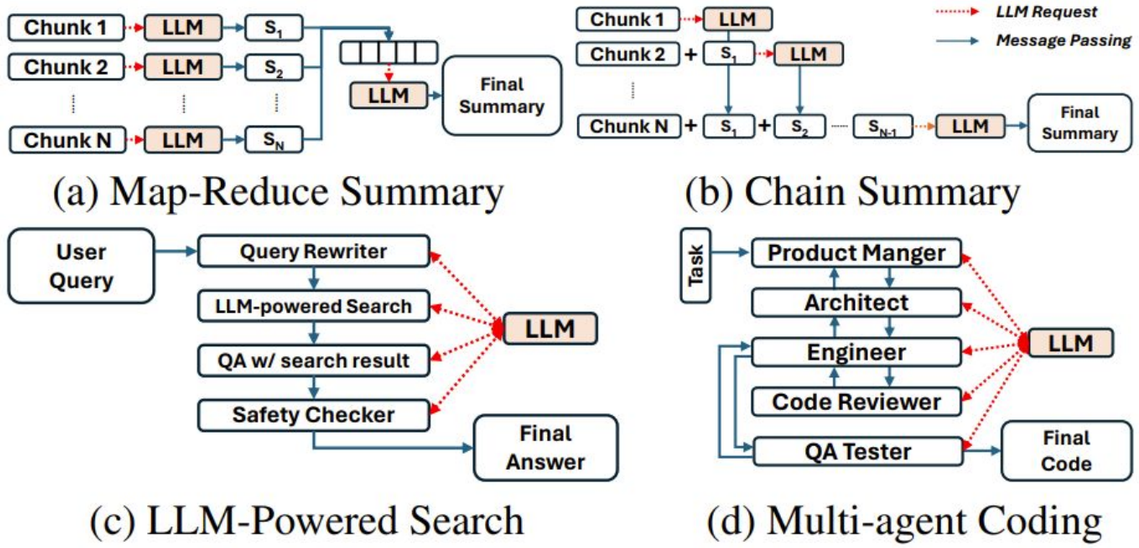
Motivation

UNIVERSITY OF MICHIGAN

Modern LLM-based applications require **multiple sequential or parallel requests** to do complex tasks.

Examples:

- **Map-Reduce Summary**
- **LLM-powered Search**
- **Chain Summary**
- **Multi-Agent Programming**



Main issue: Current serving systems **treat each request independently**, causing inefficiency.

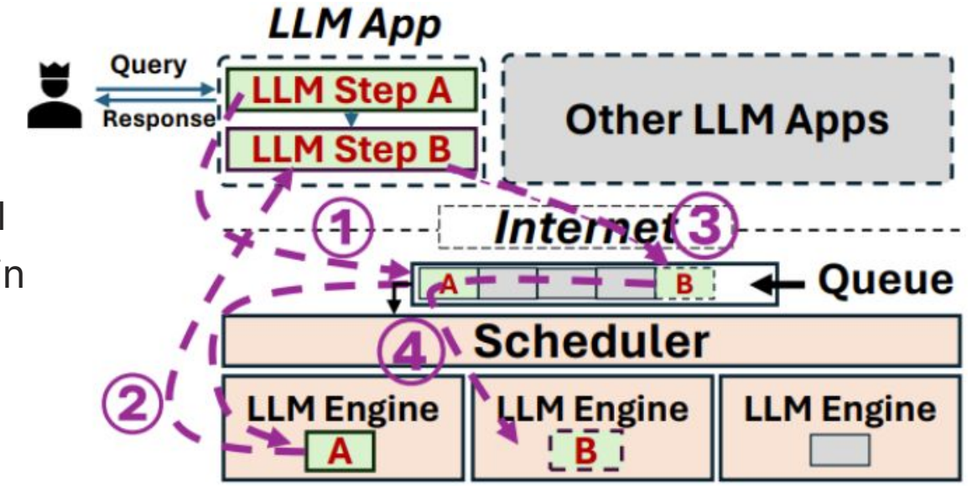
Problem Statement (1/3)

UNIVERSITY OF MICHIGAN

Main issue: Current serving systems **treat each request independently**, causing inefficiency.

Problem 1: Excessive Overhead of Consecutive Request

Author's observation: a significant portion of the latency of LLM API call originates outside the LLM engine (30~50% on average and over 70% in the worst cases).



Problem Statement (2/3)

UNIVERSITY OF MICHIGAN

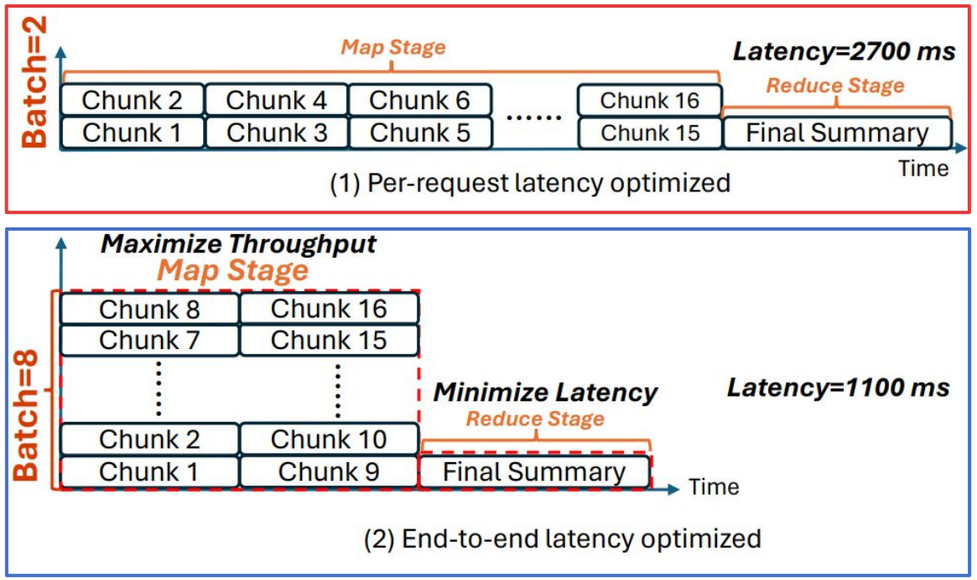
Main issue: Current serving systems **treat each request independently**, causing inefficiency.

Problem 2: Misaligned Scheduling Objectives

For example, in a Map-Reduce scenario, (ideally) the Map stage should pursue high throughput, and the Reduce stage should pursue low latency.

Existing services cannot distinguish between these two requests, it will limit the batch size on each LLM engine to control the latency of individual requests.

Example: Map-Reduce Style Doc Summary Tasks



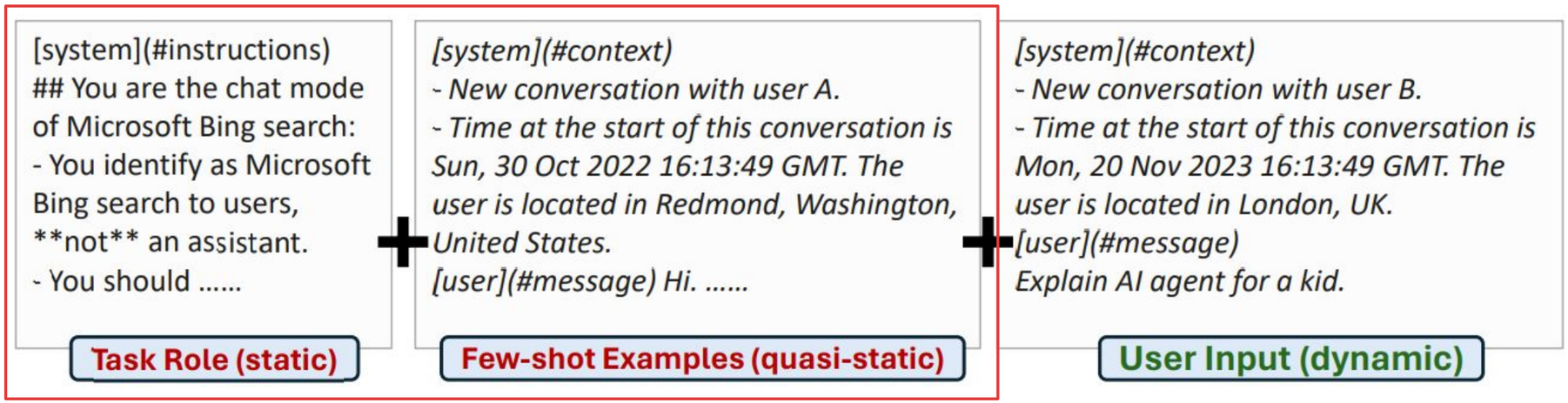
Problem Statement (3/3)

UNIVERSITY OF MICHIGAN

Main issue: Current serving systems **treat each request independently**, causing inefficiency.

Problem 3: Redundant Computations (of prefix)

Author's observation: Over 94% of tokens in the requests are repeated across different users.

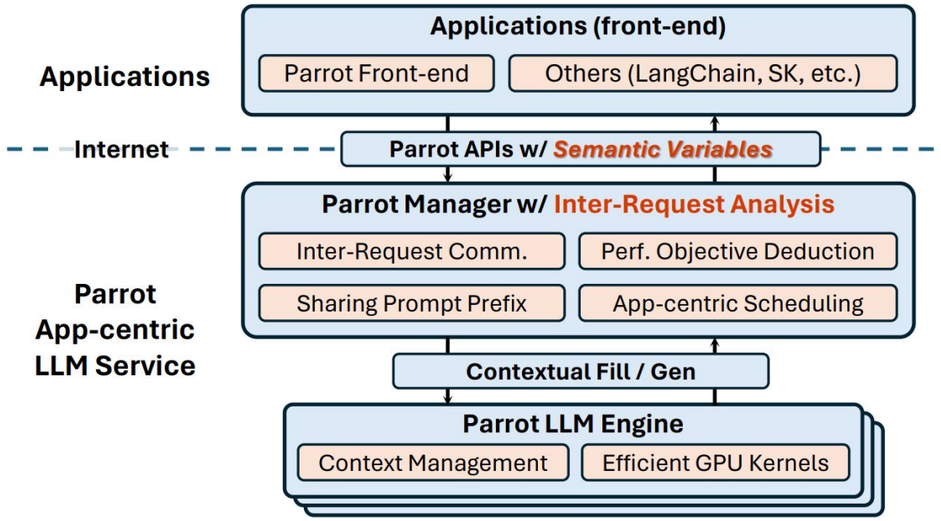


Solution: From Request-Centric to Application-Centric

Existing Systems: Treat each request independently, unaware of overall workflows.

Shift of Paradigm:

- Request-centric → Application-centric
- Individual requests → Structured requests flow



Parrot: Make the serving system "understand" application semantics.

- Introduce **semantic variables** to enable LLM services get semantic understanding.
- Leverage semantics for runtime end-to-end efficiency optimization.

Introducing Semantic Variable (SV)

Definition of SV: An input or output variable of a Semantic Function (SF), which is referred as a placeholder in the prompt. ({{input:...}} = input SVs. {{output:...}} = output SVs.)

To explain the example:

Call SF (WritePythonCode, WriteTestCode) = **Submit** request with the prompt + input SVs.

.get = fetch the value of an output SV in an on-demand manner.

This means that the server can receive all requests at once, constructing a **DAG(Directed Acyclic Graph) that represents how the requests are connected**.

```
import Parrot as P
from Parrot.PerformanceCriteria import LATENCY

@P.SemanticFunction
def WritePythonCode(task: P.SemanticVariable):
""" You are an expert software engineer.
    Write python code of {{input:task}}.
    Code: {{output:code}}
"""

@P.SemanticFunction
def WriteTestCode(
    task: P.SemanticVariable,
    code: P.SemanticVariable):
""" You are an experienced QA engineer.
    You write test code for {{input:task}}.
    Code: {{input:code}}.
    Your test code: {{output:test}}
"""

def WriteSnakeGame():
  task = P.SemanticVariable("a snake game")
  code = WritePythonCode(task)
  test = WriteTestCode(task, code)
  return code.get(perf=LATENCY), test.get(perf=LATENCY)
```

Figure 7: Example: a multi-agent application in Parrot.

Primitives of Inter-Request Analysis (1/2)

DAG-based Analysis:

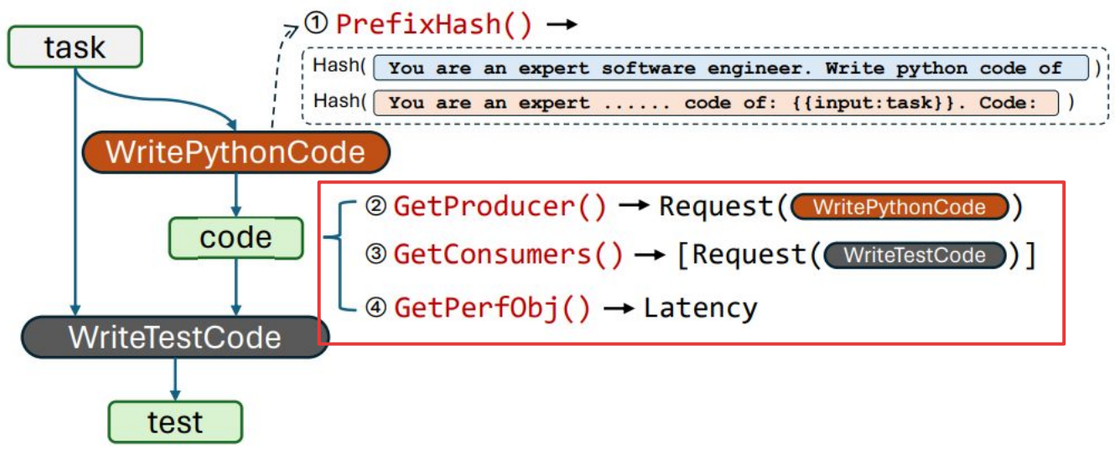
Parrot treat **each request or SV as a node**.

When a **request** comes, Parrot inserts it to DAG by **linking edges with its SVs** inside the prompts.

Parrot then perform conventional dataflow analysis using the primitives:

GetProducer / GetConsumers: Identify which requests produce or consume a given SV.

GetPerfObj: Retrieve the performance objective (e.g., latency) attached to the final output SV, which serves as the root signal for optimization.

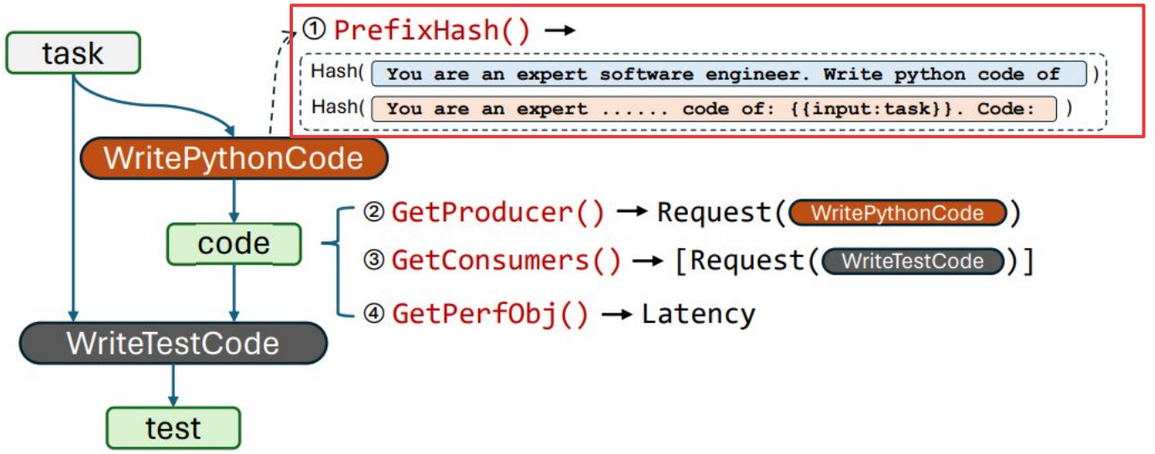


Primitives of Inter-Request Analysis (2/2)

Prompt structure-based Analysis:

Parrot extract the hash values of a request at multiple positions split by SVs(text before {{input:task}} and text before {{output:code}}).

When a new request arrives, shared opportunities can be discovered simply by checking the hash table.



Optimizations with SV (1/2)

1. Serving Dependent Requests:

With the DAG and primitives, Parrot serves dependent requests efficiently through a graph-based executor. Value transfer between nodes is accomplished through a message queue allocated for corresponding SV, avoid unnecessary communication between clients and server. Which avoid the **Problem 1: Excessive Overhead of Consecutive Request.**

2. Sharing Prompt Prefix:

With the Prompt structure-based Analysis and the hash table, Parrot can identify common prompt prefixes between different requests. If two requests are found to have the same prefix, Parrot will share the KV Cache within the same engine. Which solve the **Problem 3: Redundant Computations (of prefix).**

Optimizations with SV (2/2)

3. Performance Objective Deduction:

If a SV is labelled as “perf=throughput”, all request that generate it(directly or indirectly) inherit the throughput priority.

If labelled as "pref=latency", Parrot will traverse the DAG in reverse topological order, marking all requests that affect the final result as latency-sensitive.

```
def WriteSnakeGame():
  task = P.SemanticVariable("a snake game")
  code = WritePythonCode(task)
  test = WriteTestCode(task, code)
  return code.get(perf=LATENCY), test.get(perf=LATENCY)
```

4. Application-Centric Scheduling:

Step1 - Sort requests topologically – follow app-level dependencies.

Step2 - Group by task – batch similar-goal requests together.

Step3 - Merge shared-prefix requests – combine those with common context.

In this way, Parrot solve the **Problem 2: Misaligned Scheduling Objectives.**

Algorithm 1: Parrot’s Request Scheduling.

```
Data: Q: the request queue
Q.sort() ; /* Topological order                    */
for r ∈ Q do
    SharedReqsInQueue, CtxInEngine =
      FindSharedPrefix(r);
    if r.TaskGroup ≠ ∅ then
        r* = FindEngine(r.TaskGroup);
    else if SharedReqsInQueue ≠ ∅ then
        r* = FindEngine(SharedReqsInQueue);
    else if CtxInEngine ≠ ∅ then
        r* = FindEngine(r, filter=CtxInEngine);
    if r* = ∅ then
        r* = FindEngine(r);
    Q.remove(r*);
```

Evaluation-Experimental Setup

UNIVERSITY OF MICHIGAN

Workload

Workload	Serving Dependent Requests.	Perf. Obj. Deduction	Sharing Prompt	App-centric Scheduling
Data Analytics	✓	✓		✓
Serving Popular LLM Applications			✓	✓
Multi-agent App.	✓	✓	✓	✓
Mixed Workloads	✓	✓		✓

The workloads and the optimizations taking effect.

Testbed:

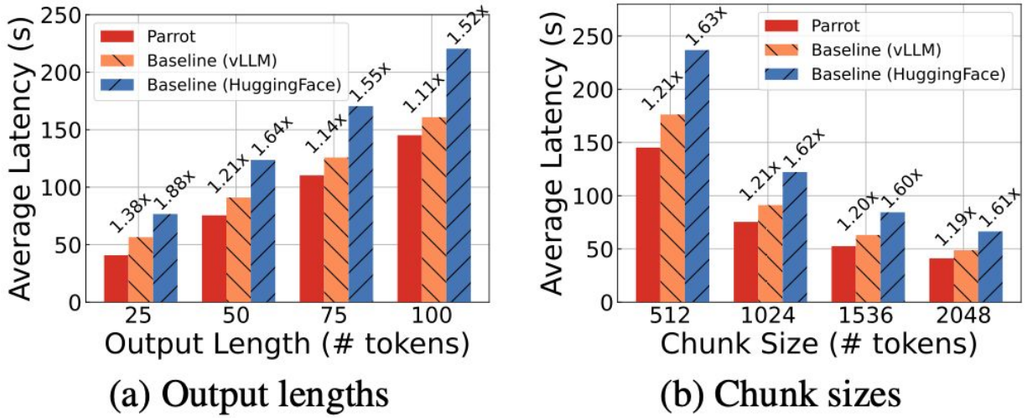
They evaluate on two servers

- Single-GPU: **24-core EPYC 7V13 + 1× A100 80GB**
- Multi-GPU: **64-core EPYC + 4× A6000 48GB**

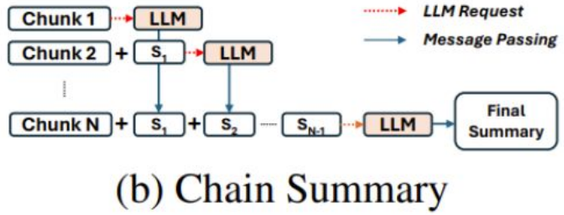
Baselines:

- **Development Framework:** LangChain
  - Helps break applications into multiple LLM calls, such as Chains or Agents.
- **Model Serving System:** FastChat
  - Handles incoming chat-style requests and dispatches them to LLM engines to HuggingFace & vLLM.
- **Inference Engine:** HuggingFace Transformers & vLLM
  - Executes model inference for LLM tasks.

Results-Data Analytics on Long Documents- Chain-style Applications



Average E2E latency of chain summarization with varying output lengths and chunk sizes.



Output length:

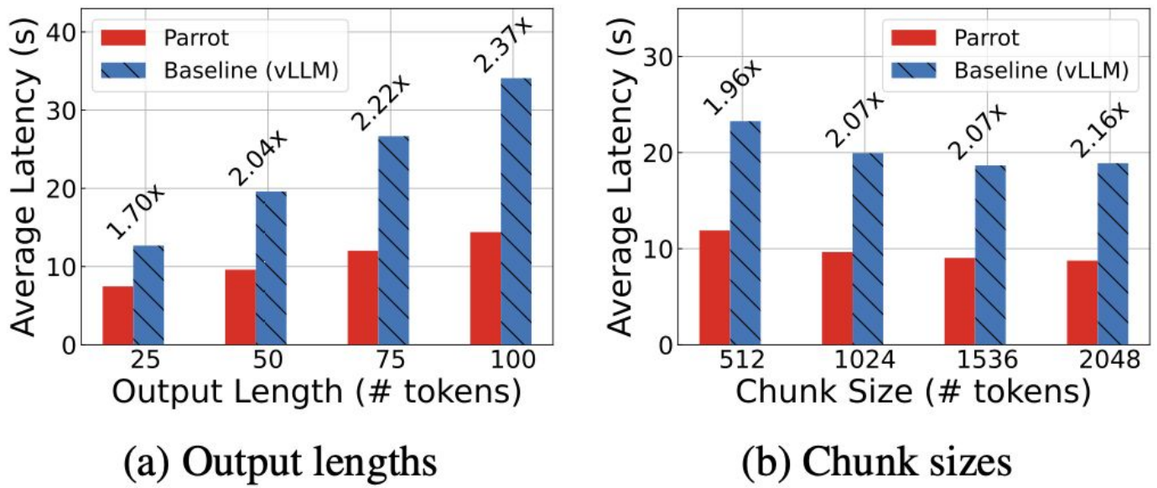
- The speedup mainly comes from fewer client–server round trips; as outputs get longer generation dominates so gains shrink
- Up to **1.38×** Speedup **vs vLLM** and **1.88×** Speedup **vs HuggingFace**.

Chunk size:

- Fewer client–server interactions and less re-queuing; however, too-large chunks raise per-step context/compute, so returns diminish.
- About **1.2× vs vLLM** and **1.66× vs HF** across chunk sizes (fixed output).

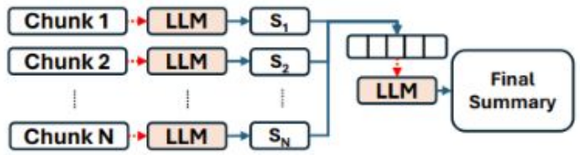
Results-Analytics on Long Documents Map-Reduce Applications

UNIVERSITY OF MICHIGAN



(a) Output lengths (b) Chunk sizes

Average E2E latency of Map-Reduce document summary with varying output lengths and chunk sizes.



(a) Map-Reduce Summary

Output Length

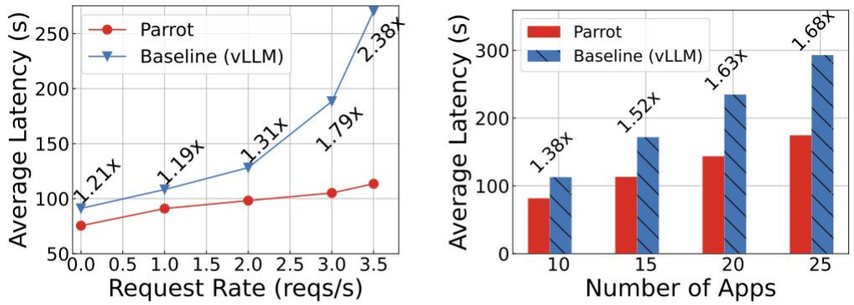
- Most of the benefit comes from reducing the number of client–server round trips. When outputs are short, communication overhead dominates, so Parrot shows large gains.
- Up to **1.38×** faster than vLLM and **1.88×** faster than HuggingFace.

Chunk Size

- Larger chunks reduce scheduling overhead and re-queuing, so fewer intermediary steps are required, lowering latency.
- **~1.2×** faster than vLLM and **1.66×** faster than HuggingFace across chunk sizes (fixed output).

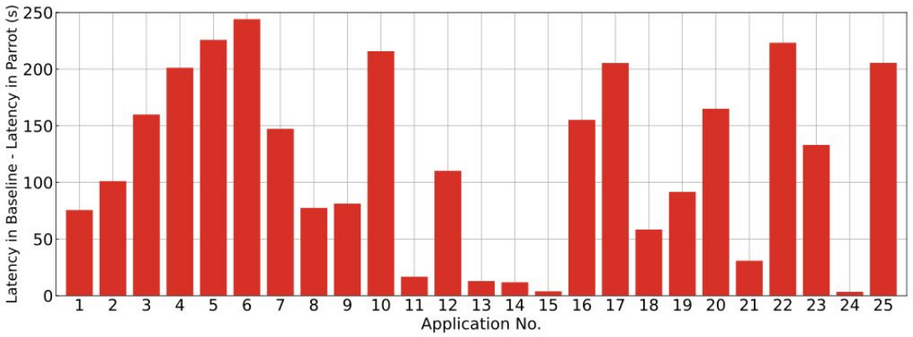
Data Analytics on Long Documents

UNIVERSITY OF MICHIGAN



(a) With background requests (b) Multiple summary apps.

Average E2E latency of chain-summary with back-ground requests or other chain-summary applications.



The difference in E2E latency of the 25 chain-summary application between Baseline and Parrot. All applications finish earlier in Parrot.

With background requests

- Parrot's curve rises slowly; baseline (vLLM) shoots up. Speedup grows from **~1.21×** at light load to **2.38×** at 3.5 req/s.

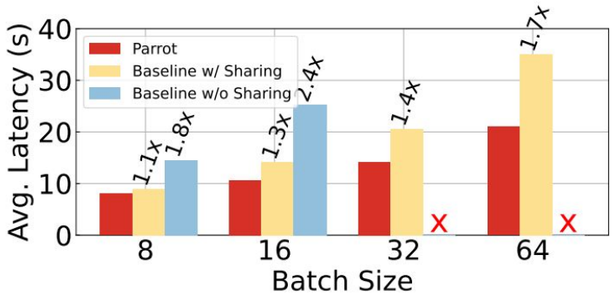
Multiple summary apps

- Mean E2E latency rises for both, but Parrot stays lower with a widening gap up to **1.68×**faster than baseline.

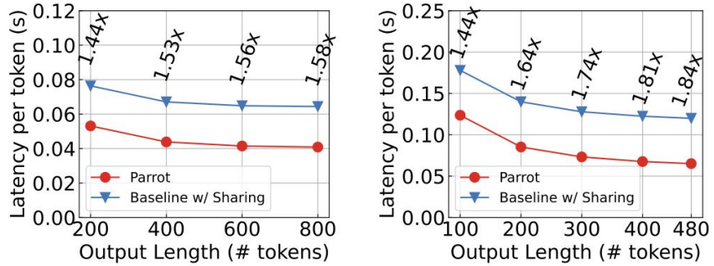
- **Takeaway:**
  a. **Serving Dependent Requests** – Keep intermediate KV/cache across chained steps instead of recomputing each stage.
  b. **Performance Objective Deduction** – Detect that the goal is low end-to-end latency (not throughput) and auto-select small batches + fast scheduling.
  c. **App-centric Scheduling** – Run all steps of the same document on the same engine to avoid cache eviction and queue delays.

Results-Serving Popular LLM Applications

UNIVERSITY OF MICHIGAN

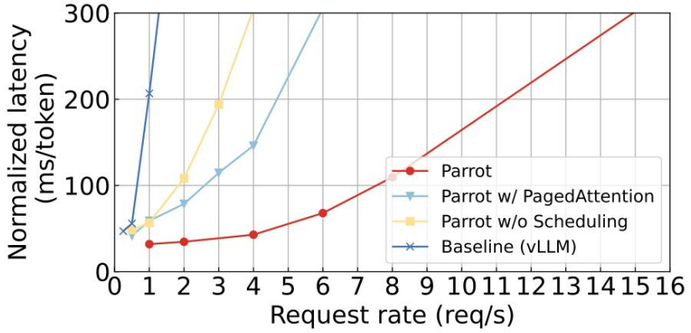


Latency of Bing Copilot with varying batch sizes.



(a) Batch Size = 32 (b) Batch Size = 64

Latency per output token of Bing Copilot.



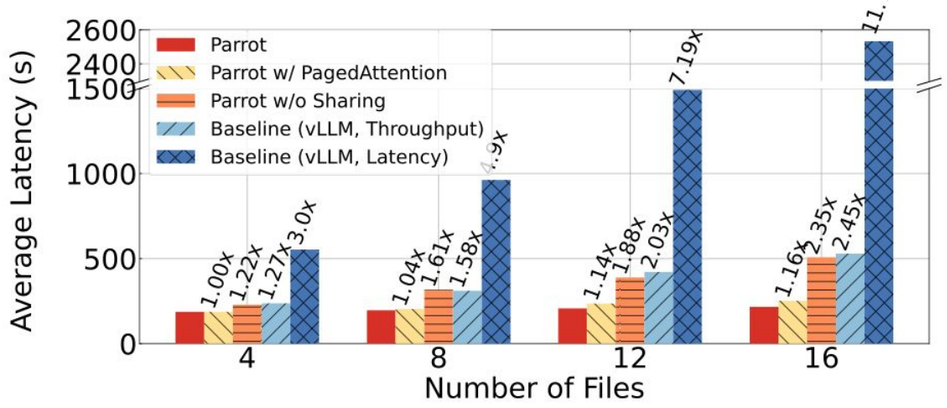
Serving multiple GPTs applications.

- Parrot keeps per-token latency low (up to **1.84×** faster than vLLM) and can sustain **12×** higher request rates in multi-application serving by scheduling requests with shared prompts onto the same engine and using a kernel that avoids redundant memory loads.

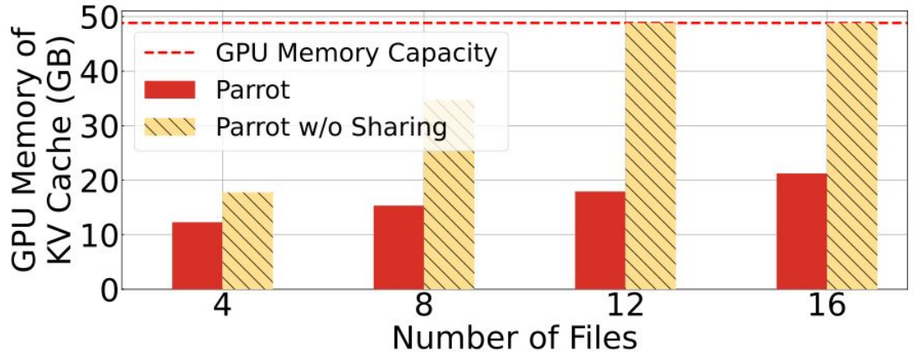
- Parrot outperforms both baselines because it avoids repeatedly reloading the shared prompt into GPU memory, achieving **1.8–2.4×** speedup over the no-sharing baseline and **1.1–1.7×** over vLLM, which shows that KV or prompt sharing alone is insufficient without an efficient kernel.

Results-Multi-agent Applications

UNIVERSITY OF MICHIGAN



(a) End-to-end Latency



(b) GPU Memory of KV Cache

The latency and memory usage for multi-agent programming, with varying number of files to program.

Architect → Coders → Reviewers → Coders revise → Final code output.

End-to-end Latency.

- Parrot is up to **11.7× faster** than the latency-centric baseline.
- Parrot is up to **2.45× faster** than the throughput-centric baseline.

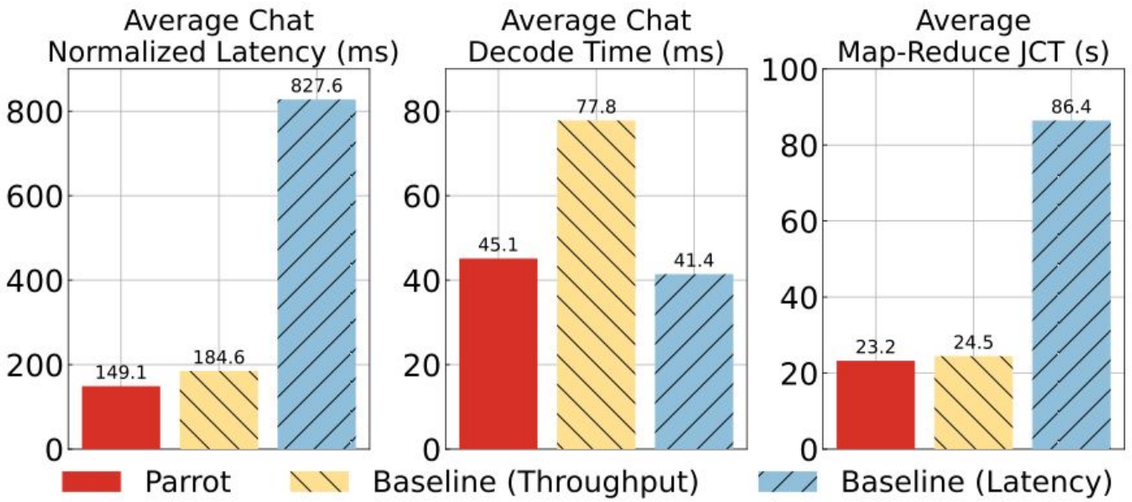
KV Cache Memory Usage

- Parrot keeps memory around **18 GB**, staying safely below the GPU limit even at 16 agents.

- **Takeaway:** Parrot's prompt-structure analysis eliminates redundant context work, worth about 2.35× of that gain; its shared-prefix kernel adds another 1.2× at 16 files.

Results-Scheduling of Mixed Workloads

UNIVERSITY OF MICHIGAN



The mixture of chat and map-reduce applications.

- **Chat latency:** Parrot cuts normalized chat latency by up to **5.5×** compared to the latency baseline.
- **Chat decode speed:** Parrot matches the latency-oriented baseline and is **1.7× faster** than the throughput baseline.
- **Map-Reduce throughput:** Parrot completes Map-Reduce jobs **3.7× faster** than the latency baseline while keeping throughput on par with the throughput baseline.

Takeaway: Parrot is the only system that delivers both low-latency chat and high-throughput Map-Reduce, avoiding the trade-off that forces traditional systems to choose one and sacrifice the other.

UNIVERSITY OF MICHIGAN

Towards End-to-End Optimization of LLM-based Applications with Ayo

Xin Tan, Yimin Jiang, Yitao Yang, Hong Xu

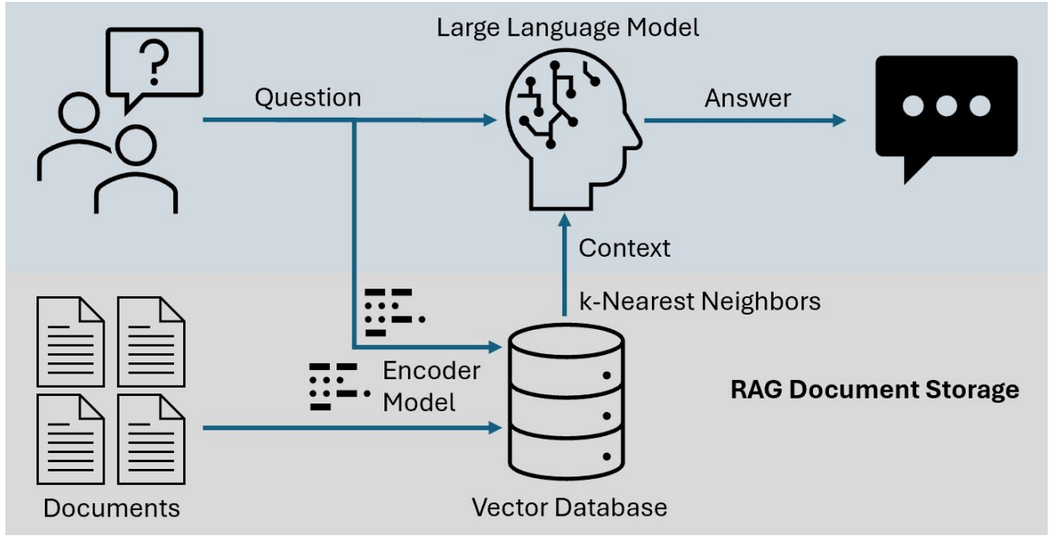
The Chinese University of Hong Kong & Independent Researcher

ASPLOS 2025 | April 2025

TL;DR

UNIVERSITY OF MICHIGAN

- Mostly, current research focuses narrowly on improving the model inference itself
  - KV Cache reuse, Batching, etc
- Ignores the rest of the end-to-end application efficiency
  - API Call, RAG retrieval, etc
- Ayo optimize the **end to end efficiency** by primitive level orchestration and correspondence system level optimization

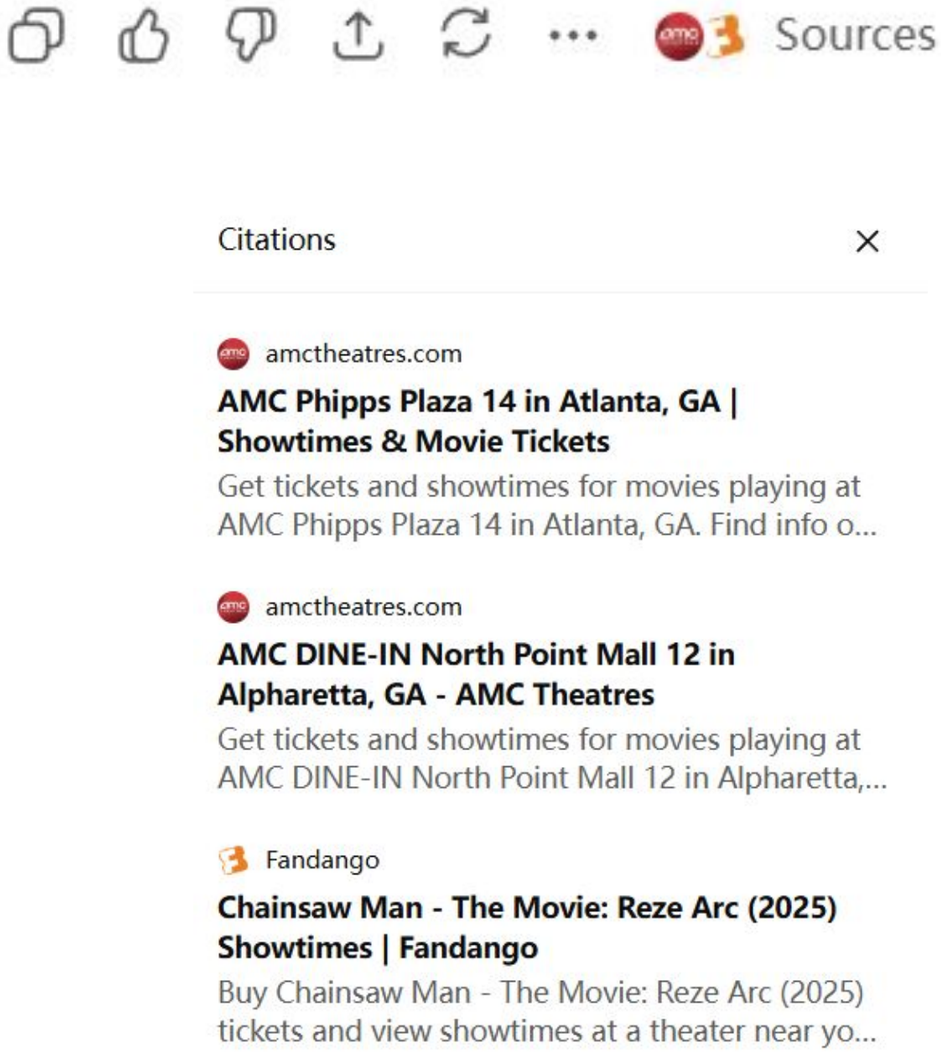


Intro:LLMs are more than LLMs

UNIVERSITY OF MICHIGAN

- Currently, LLM shows two key limitations
  - The training data is always out of date
  - Lack of ability to interact with environments
- Very large demand for such systems
- *SE stands for search engine*
- *Results gained from survey high star LLM repo*

Workflow	Count	Proportion
SE generation	16	53.33%
LLM agent	13	43.33%
Doc QA w/ naive RAG	26	86.67%
Doc QA w/ advanced RAG	23	76.67%



The Problem with Current Orchestration Frameworks

Coarse-grained Orchestration

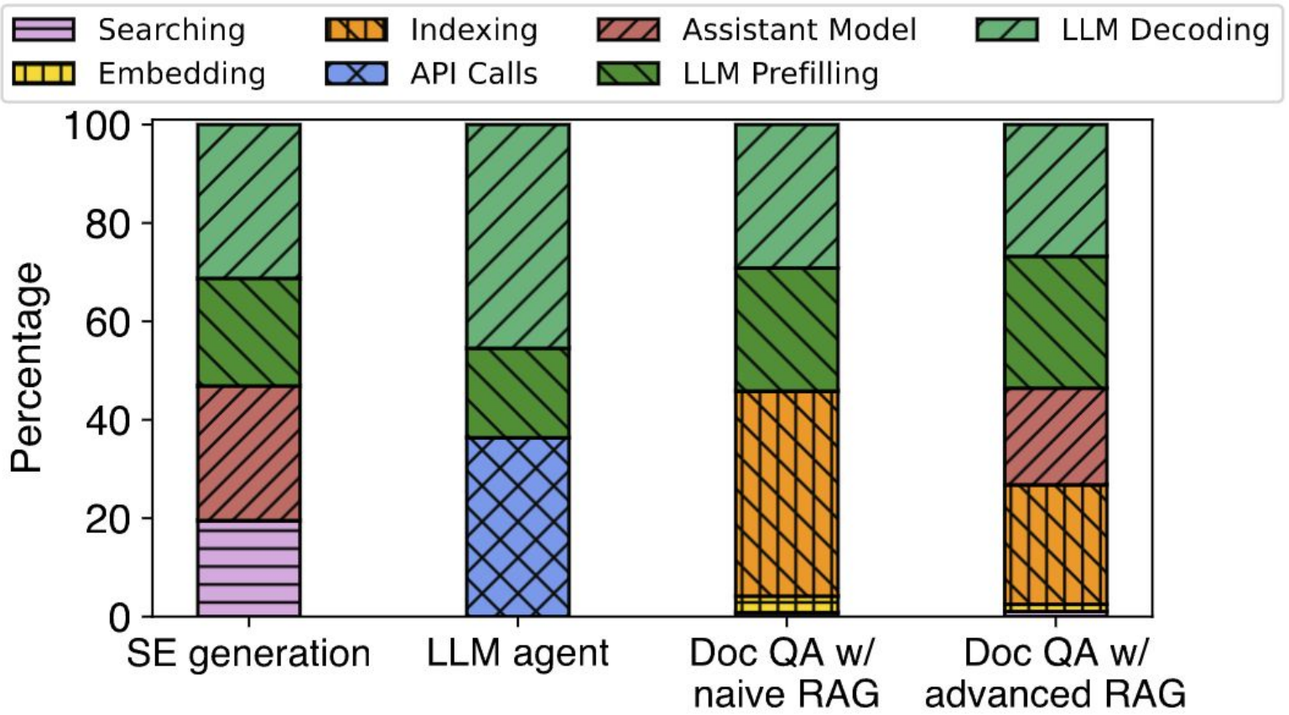
- Popular frameworks like LangChain organize applications in **sequential order**

Task Agnostic Backend

- Inference engine is task agnostic.
- Can’t identify opportunities for parallelization and batching.

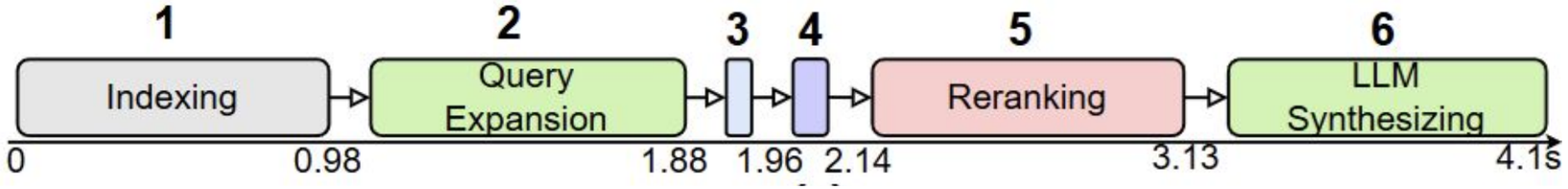
Scheduling Inefficiency

- Existing systems schedule requests through the pipeline
- Missing opportunities to optimize across requests

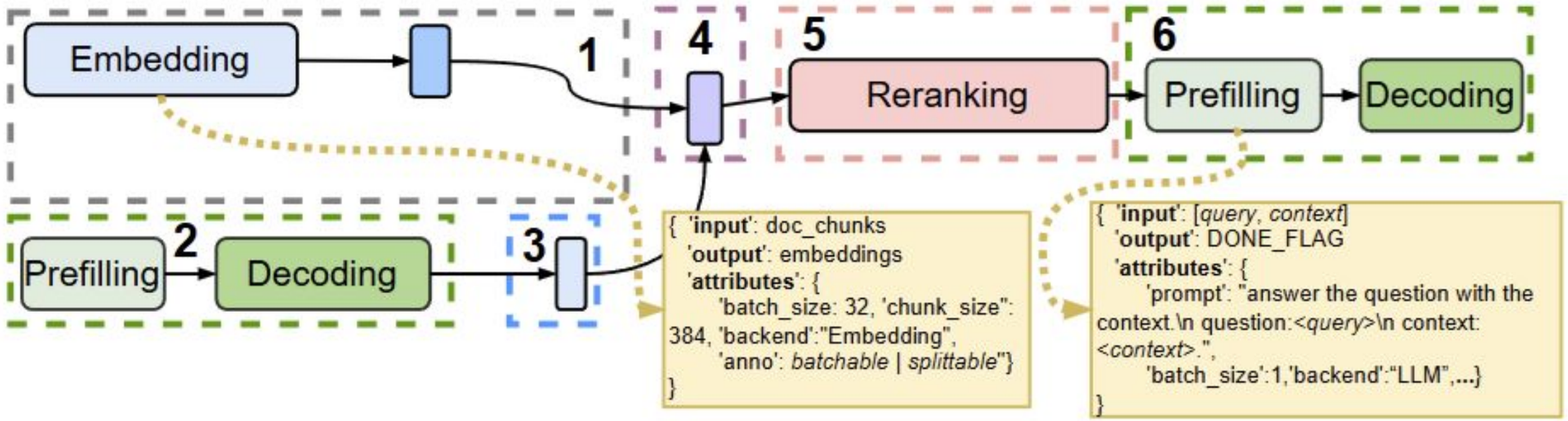


Example-Coarse-grained Orchestration

UNIVERSITY OF MICHIGAN



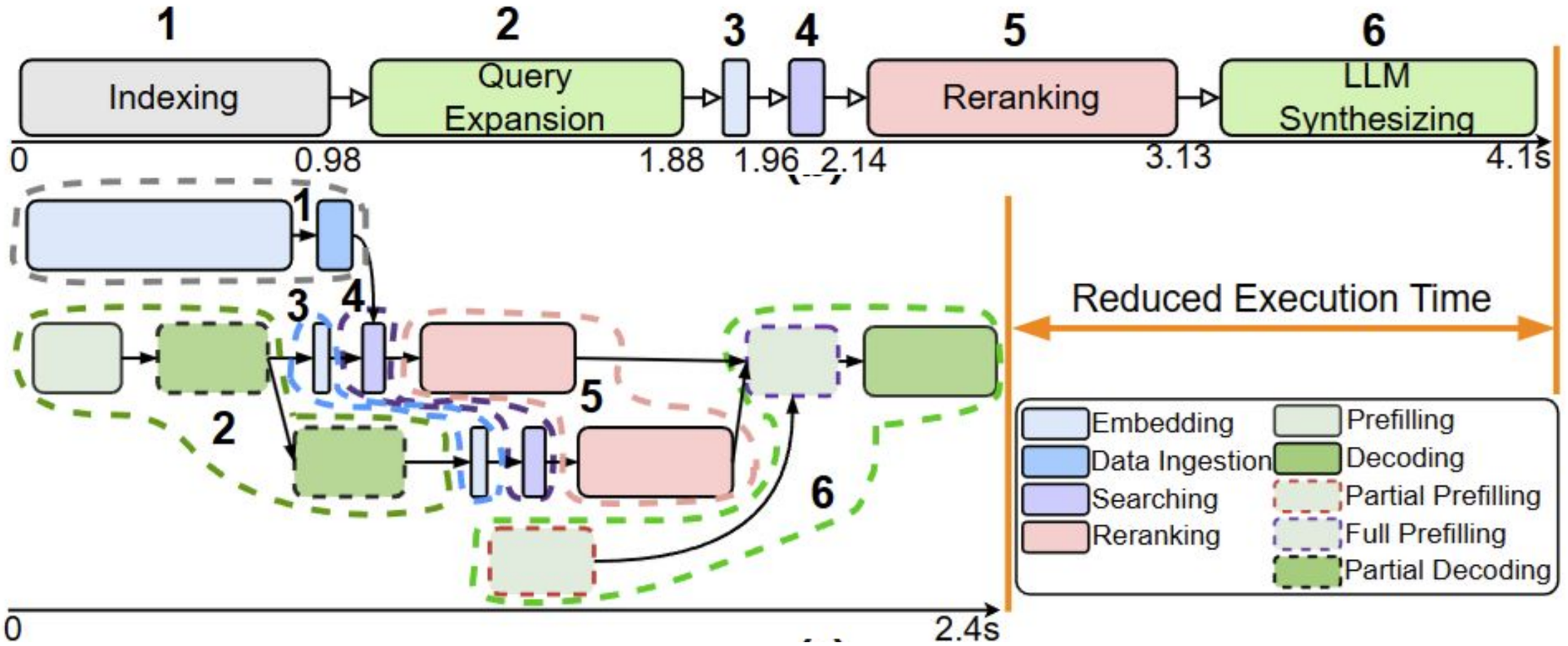
- An example of RAG workflow in LangChain
- Data Dependency Graph:



Example-Coarse-grained Orchestration

UNIVERSITY OF MICHIGAN

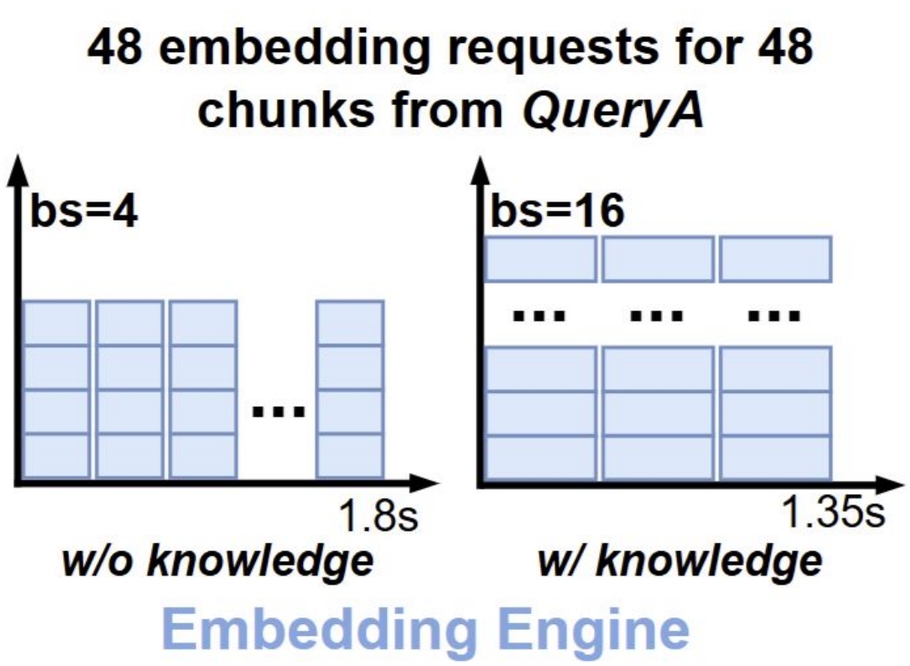
- The data ingestion and embedding can run parallel
- Expand query can run parallel
- Partial prefill for system prompt

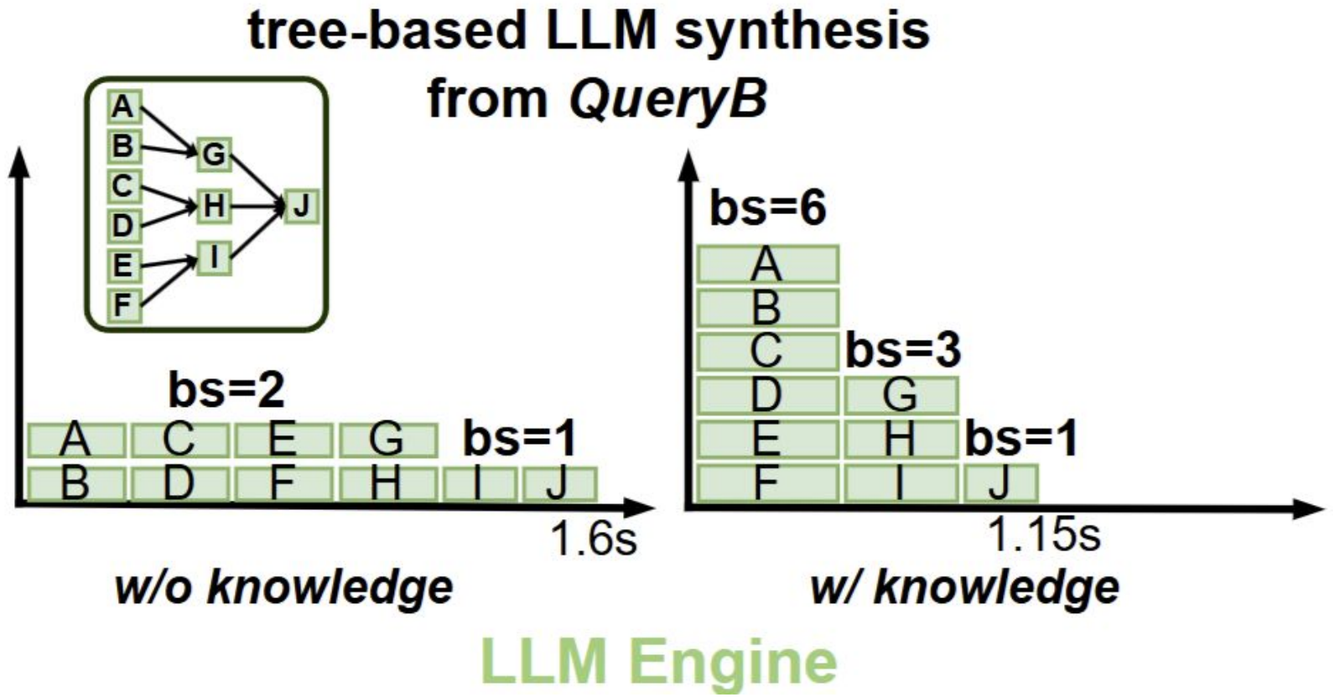


Example-Task Agnostic Backend

UNIVERSITY OF MICHIGAN

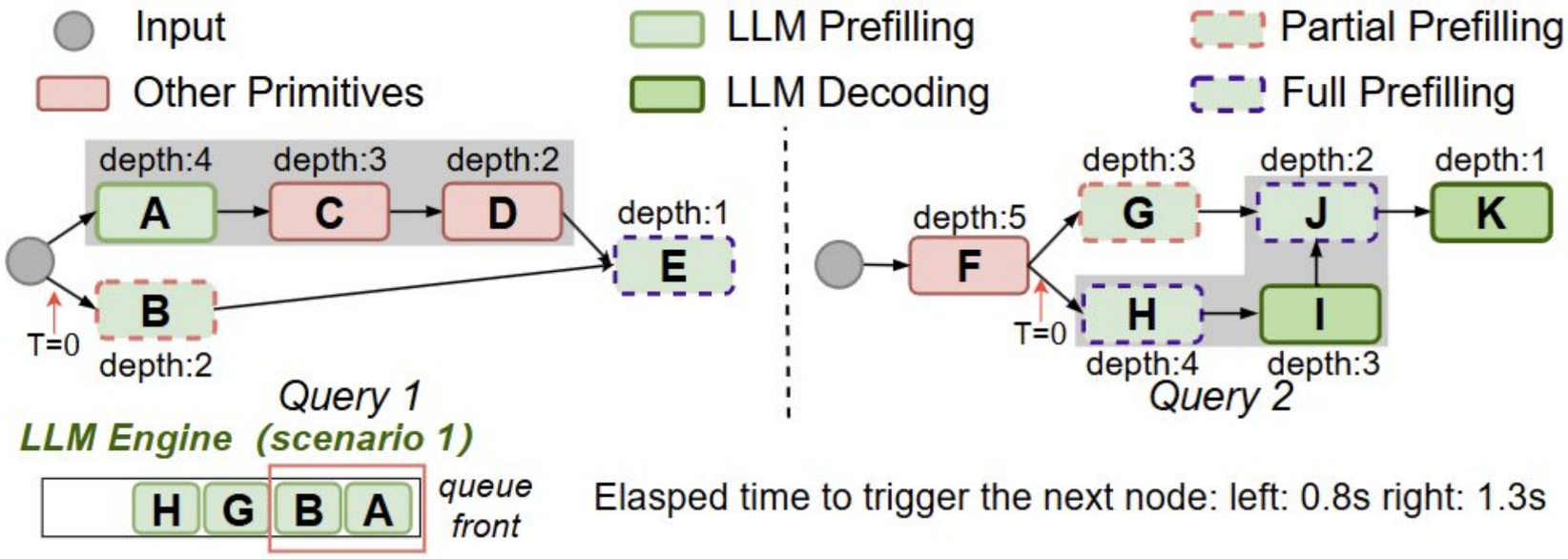
- Consider two examples





Scheduling Inefficiency

UNIVERSITY OF MICHIGAN



(a) Blind batching.

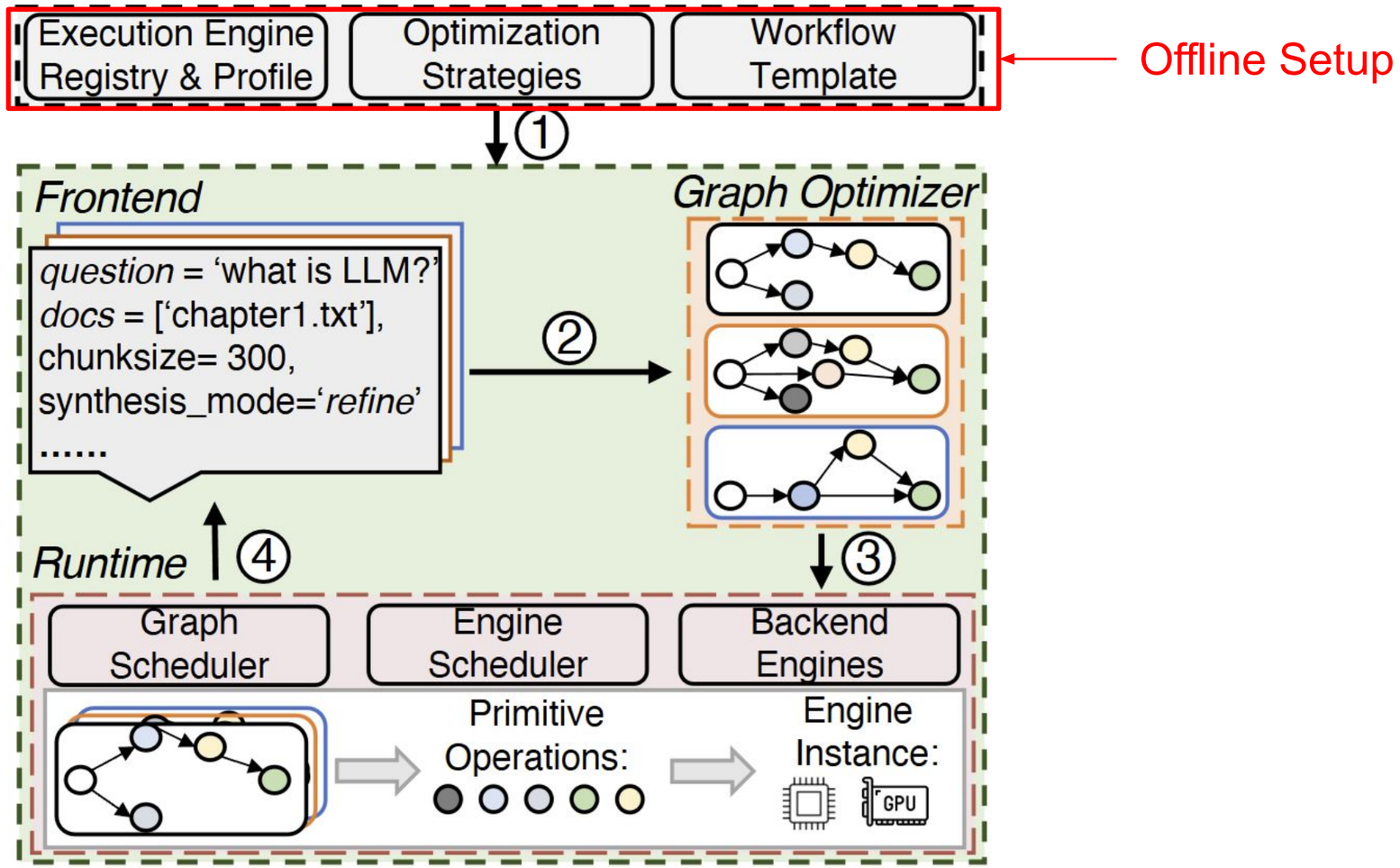
Summary & Recap

UNIVERSITY OF MICHIGAN

- Current systems most focus on “unit level optimization”
  - Flash Attn, DeepSpeed, etc
- However, holistic optimization remains coarse grained
  - LangChain execute tasks mostly sequentially
- Scheduling across request remains inefficient
  - Tasks without dependencies across request can’t run in parallel
- The holistic optimization can be decomposed into three:
  - Executing graph should be optimized
  - Execute Engine should be task aware
  - Dependency aware queueing algorithm

Solution Overview

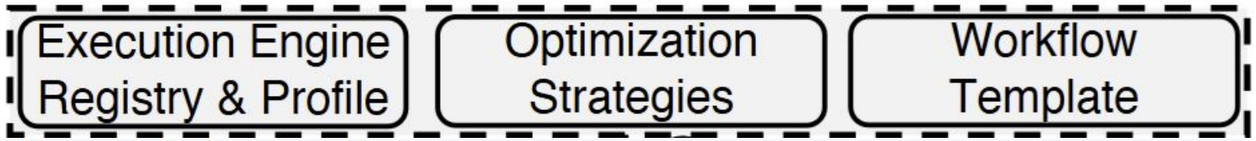
UNIVERSITY OF MICHIGAN



Offline Setup

UNIVERSITY OF MICHIGAN

- Register nodes
  - Name
  - Config
  - ...
- Form a DAG
- Perform Optimization
- Profiling the system capacity




```
llm_prefilling_node = Node(
    name="LLMPrefilling",
    node_type=NodeType.COMPUTE,
    engine_type=EngineType.LLM,
    io_schema=NodeIOSchema(
        input_format={"queries": List[str], "reranked_results": List[List[str]]},
        output_format={"prefill_state": bool}
    ),
    op_type=NodeOps.LLM_PREFILLING,
    config={
        'prompt_template': replace_placeholders(RAG_QUESTION_ANSWERING_PROMPT_TEMP
        'parse_json': True,
        #other config ...
    }
)
# create the DAG
dag = DAG(dag_id="rag_workflow")
dag.register_nodes(llm_prefilling_node, llm_decoding_node, ...)
```

```
dag.optimize([PruningDependencyPass(), StageDecompositionPass(), PrefillingSpiltPass(), LLMDecodingPipe
```

Graph Optimizer

UNIVERSITY OF MICHIGAN

Algorithm 1 Graph transformation and optimization

```
1:  function GraphTransform(𝒯, C)
2:      𝒱_N ← {}; 𝒱_E ← {} ▷ primitives and their data dependency into
        # Decompose each template component with configuration into
        # a sub-graph with primitives and maintain sub-graph dependency
3:      for each t ∈ 𝒯_N do
4:          Prims, Edges ← DecomposeComponent(t, C)
5:          Prims ← Configure(Prims, C)
6:          𝒱_N.extend(Prims); 𝒱_E.extend(Edges)
        # Maintain template's original component dependency
7:      for each (t_i, t_j) ∈ 𝒯_E do
8:          tailp ← GetTailPrim(t_i); headp ← GetHeadPrim(t_j)
9:          𝒱_E.append((tailp, headp))
10:     return 𝒢_p = (𝒱_N, 𝒱_E)          ▷ return primitive-level p-graph
11: function GraphOpt(𝒢_p, 𝒫)
        # 𝒢_p for p-graph, 𝒫 for profile of execution engines
12:     𝒢_e ← PrunDependency(𝒢_p)                ▷ Pass 1
13:     𝒢_e ← StageDecompose(𝒢_e, 𝒫)             ▷ Pass 2
14:     𝒢_e ← PrefillingSplit(𝒢_e)               ▷ Pass 3
15:     𝒢_e ← DecodingPipeling(𝒢_e)              ▷ Pass 4
16:     return 𝒢_e                    ▷ return optimized e-graph
```

← Construct Dependency

← 4 Pass Optimization

Definition of "primitive" is skipped for simplicity

Four Powerful Optimization Techniques

UNIVERSITY OF MICHIGAN

Dependency Pruning

- Delete edges that don’t contain data dependencies

Stage Decomposition

- Breaks operations into smaller micro-batched stages that can be pipelined.
- Allows downstream stages process partial results before complete entirely

LLM Prefilling Split

- For requests that arriving different times, find the best scheduling for prefill
- Prefill system prompts first, etc

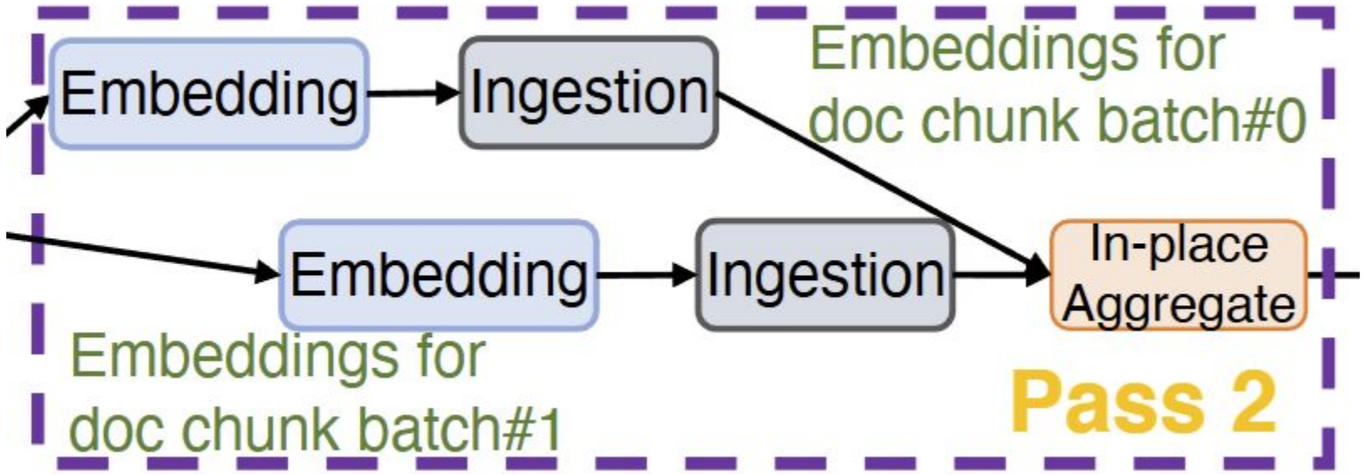
LLM Decoding Pipeline

- Identify the downstream dependencies
- Stream the partial sequence to avoid full sequence delay
- Json outputs or math expressions

Stage Decomposition - Pass 2

UNIVERSITY OF MICHIGAN

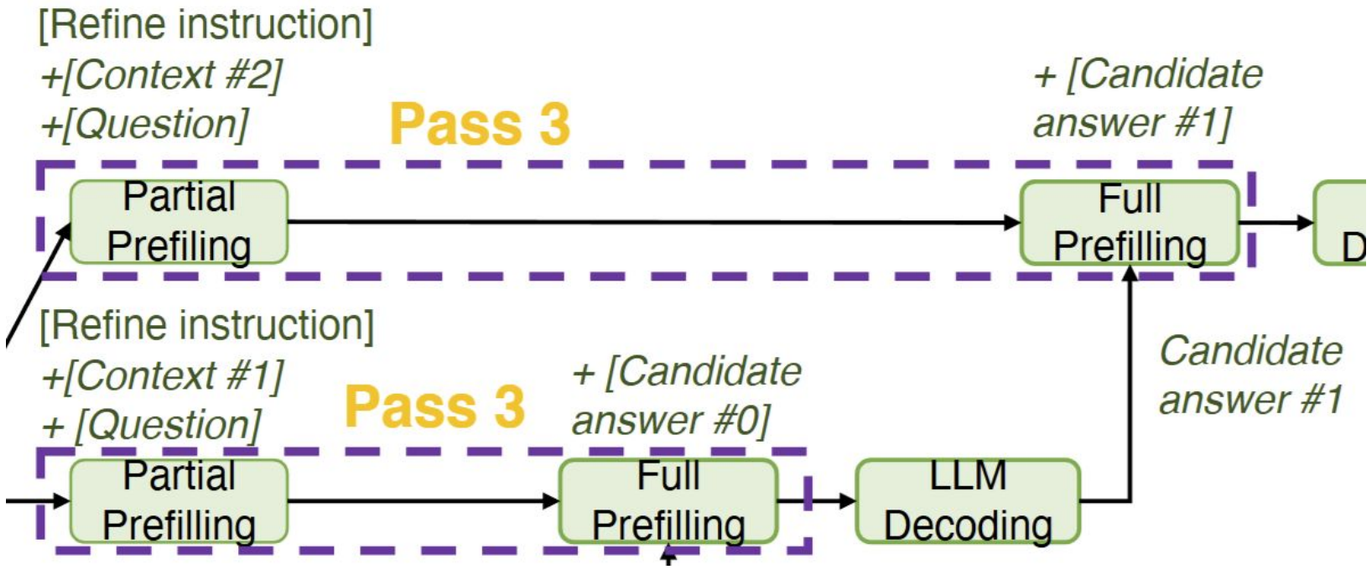
- Select “Best” batch size according to inference engine and hardware
  - Inference engine’s optimal batch size is 1024 (from profile)
  - Consider a document with 3K tokens (> optimal batchsize)
- The batchsize will be 1024 and document will be split into **three**
- The pipeline will be split into three



LLM Prefilling Split - Pass 3

UNIVERSITY OF MICHIGAN

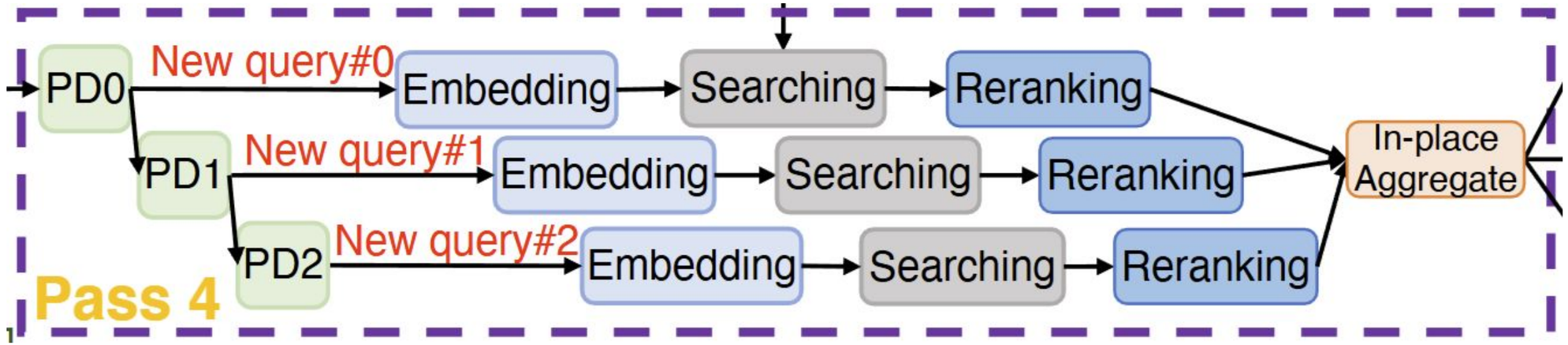
- Start prefill early for inputs that is determined (no matter length)
- Such as system prompt or questions
- When the retrieved text was back, finish the rest of input
- For short input, this may slow down the execution, but with **greater end to end gain**
- Modified vllm to support partial prefill



LLM Decoding Pipeline - Pass 4

UNIVERSITY OF MICHIGAN

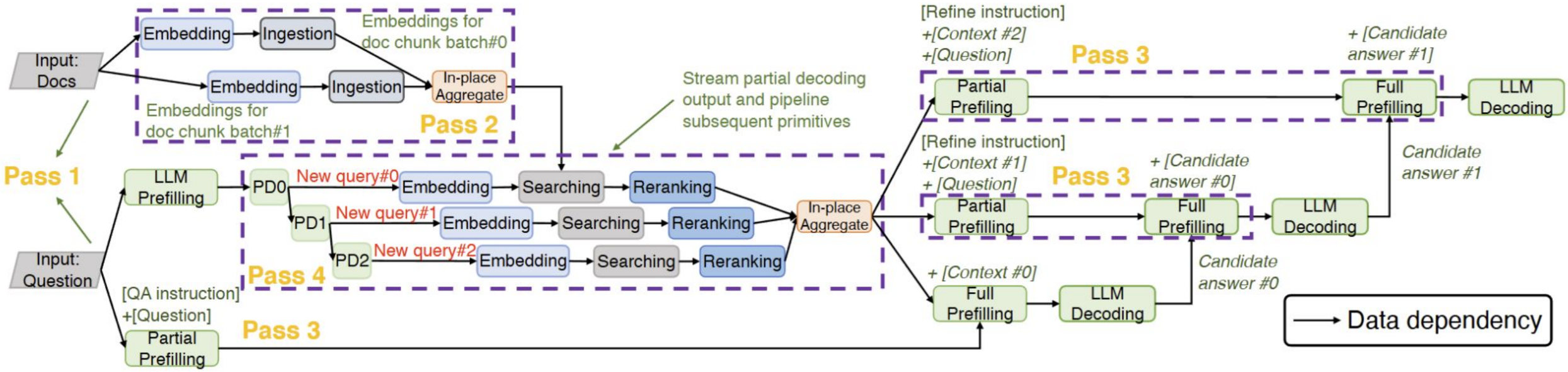
- Before the decoding start, this execution will be tagged as “splittable”
- Start partial downstream task when conditions met
  - Such as one query done, directly send to query
- Rules:
  - The downstream task is splittable
  - The partial sequence is enough for downstream pipeline



Optimized Graph

UNIVERSITY OF MICHIGAN

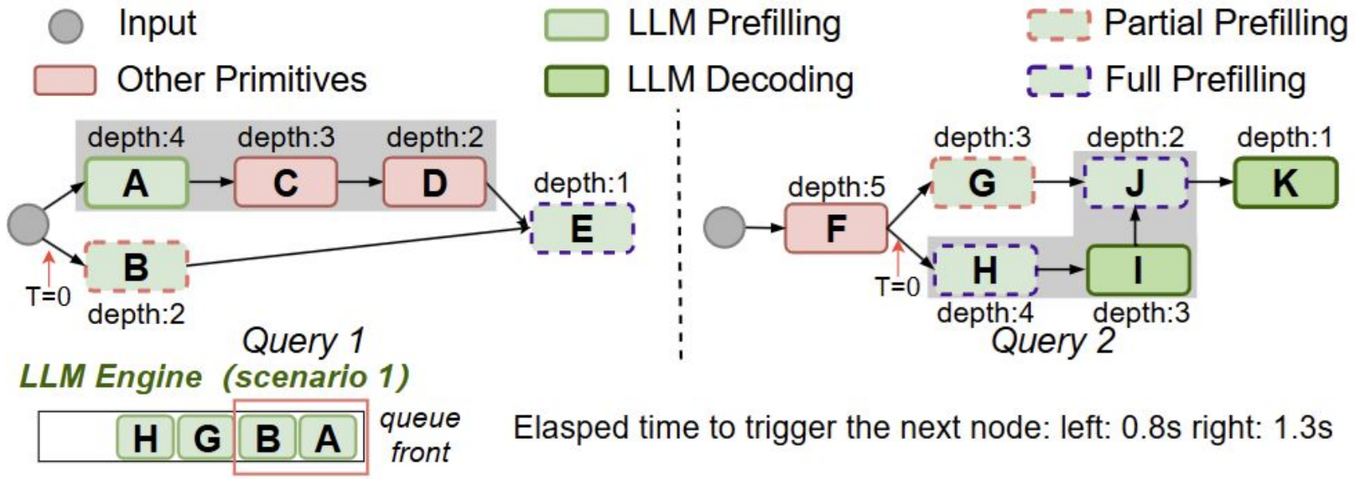
- RAG-based document QA with refine synthesis mode
- PD means partial decoding here



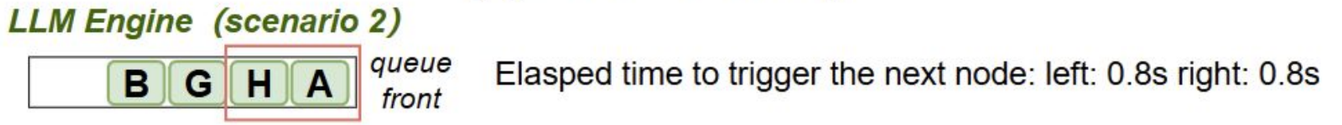
Runtime Optimization - Graph Level

UNIVERSITY OF MICHIGAN

- **Topology depth priority**
  - Ayo maintain a queue across graphs
  - Higher depth (topology depth) nodes has higher priority
  - In paper, they calculate this recursively
- **Batch compatibility grouping**
  - Same task from different graph will be execute together (e.g. embedding)



(a) Blind batching.



(b) Topology-aware batching.

Runtime Optimization - Engine-Level

UNIVERSITY OF MICHIGAN

- Different requests but share same engine is allowed to run at same time
  - E.g a synthesis and a generation but same LLM
- instance load balance
  - Balance instance load with KV Cache usage(LLM) or request number
- Fault tolerance
  - Caching node results in local storage

Experimental Setup

UNIVERSITY OF MICHIGAN

Model	Deployment
Gemma-2-2B	1×3090
Llama-2-7B	1×3090
Llama-2-13B	2×3090
Llama-2-30B	2×A800

Baselines:

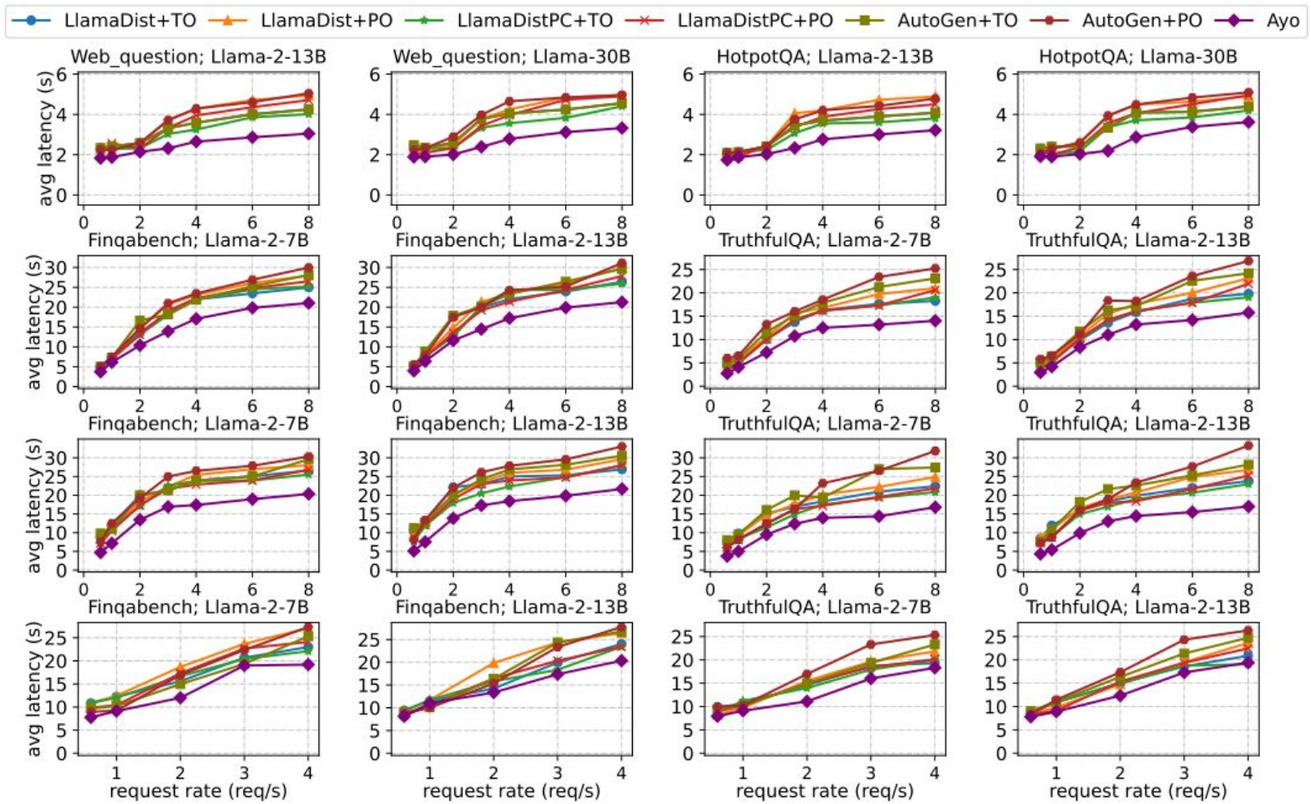
- LlamaDist
- LlamaDistPC (parallel + cache-reuse)
- AutoGen:
- Per-Invocation oriented (PO)
- Throughput oriented (TO)

Tasks:

- Search engine-empowered generation
- Document QA with naive RAG
- Document QA with advanced RAG
- Contextual retrieval

End-to-end Performance

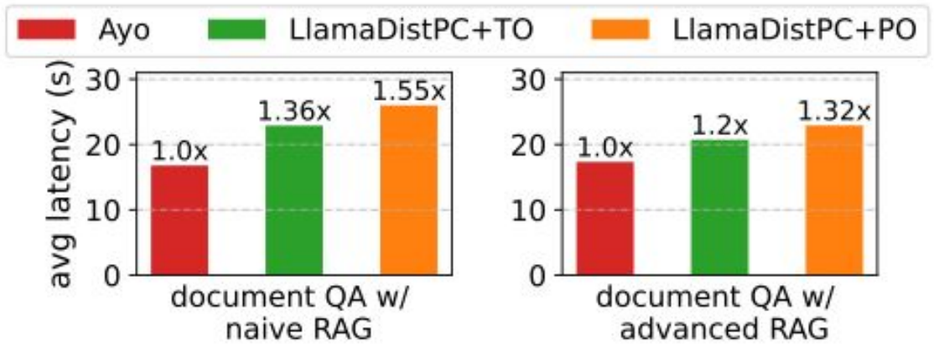
UNIVERSITY OF MICHIGAN



Workload	Ayo Speedup
Search engine-empowered generation	**Up to 1.79× faster**
Document QA with naive RAG	**1.62× (low load) – 1.67× (high load)**
Document QA with advanced RAG	**Up to 2.09× (low load) / 2.03× (high load)**
Contextual retrieval	**1.06× – 1.59×**

More results-Co-located Applications

UNIVERSITY OF MICHIGAN



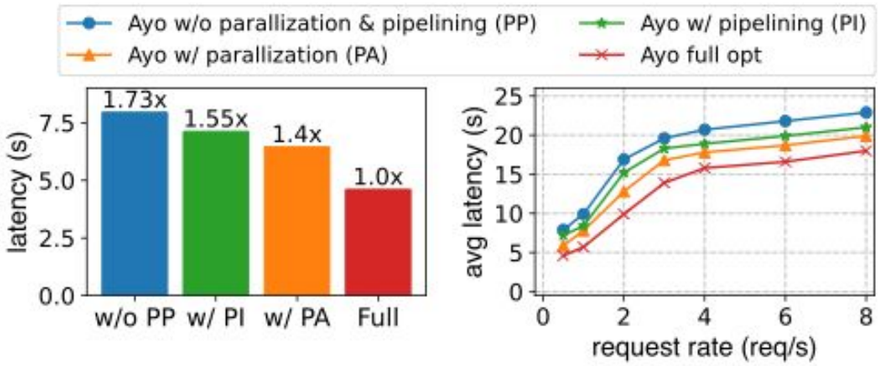
The average latency for two apps under a co-location scenario using llama-2-13B as the core LLM on TruthfulQA dataset.

- Tested multiple apps share the same LLM backend but have different workflows
- Ayo delivers **1.2×–1.55×** lower latency than the baseline under co-location.

- **Takeaway:**Reuses shared work and schedules apps smartly, avoiding duplicate compute and queueing delays.

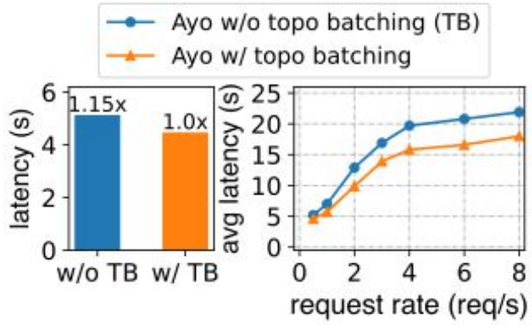
More results-Ablation Study

UNIVERSITY OF MICHIGAN

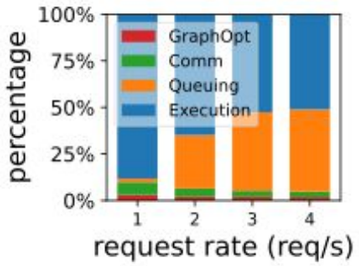


Left: single-query latency averaged over 10 runs. Right: average latency under varying request loads.

Ablation study on graph optimization in document QA with advanced RAG on truthfulQA dataset using llama-30B as core LLM.



Ablation study on runtime scheduling.



Latency breakdown of Ayo's execution critical path.

- Both techniques matter, and they stack *parallelization + pipelining* gives the full speedup.
- *Graph optimizations* help not just single queries but also high-concurrency scenarios.
- *Topology-aware batching* cuts single-query latency by **1.15×** and delivers even bigger gains under load by removing redundant work across concurrent queries.
- Ayo's speedup comes from shortening both *execution* and *waiting* time, not just compute.

Summary Comparison

UNIVERSITY OF MICHIGAN

	Parrot (OSDI ’24)	Ayo (ASPLOS ’25)
Core Problem	Current **serving systems treat each request independently**, causing inefficiency.	Lack of **holistic** view of LLM application optimization **Coarse grained orchestration** Such as LangChain, it runs sequentially
Core Idea	**Introduce semantic variables in each request to let scheduler get semantic understanding of the requests, then schedule requests and reuse prefix cache.(requests)**	**Transit workflow into a graph** **Optimize on the graph to reduce dependency** **Enable parallel and micro pipeline execution(workflow)**
Optimization Target	**Requests scheduling and redundant computation across requests**	**Sequential execution within a single request workflow**